

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

PRAKRITHI JAIN (1BM23CS239)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Sep-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Prakrithi Jain (1BM23CS239)**, who is bonafide student of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Prof. Swathi Sridharan Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
---	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	28/08/2025	Genetic Algorithm for Optimization Problems	4-17
2	11/09/2025	Particle Swarm Optimization for Function Optimization	18-29
3	09/10/2025	Ant Colony Optimization for the Traveling Salesman Problem	30-42
4	16/10/2025	Cuckoo Search (CS)	42-49
5	23/10/2025	Grey Wolf Optimizer (GWO)	50-58
6	30/10/2025	Parallel Cellular Algorithms and Programs	59-67
7	04/09/2025	Optimization via Gene Expression Algorithms	68-78

Github Link:

https://github.com/prakrithi726/1BM23CS239_BIS-LAB.git

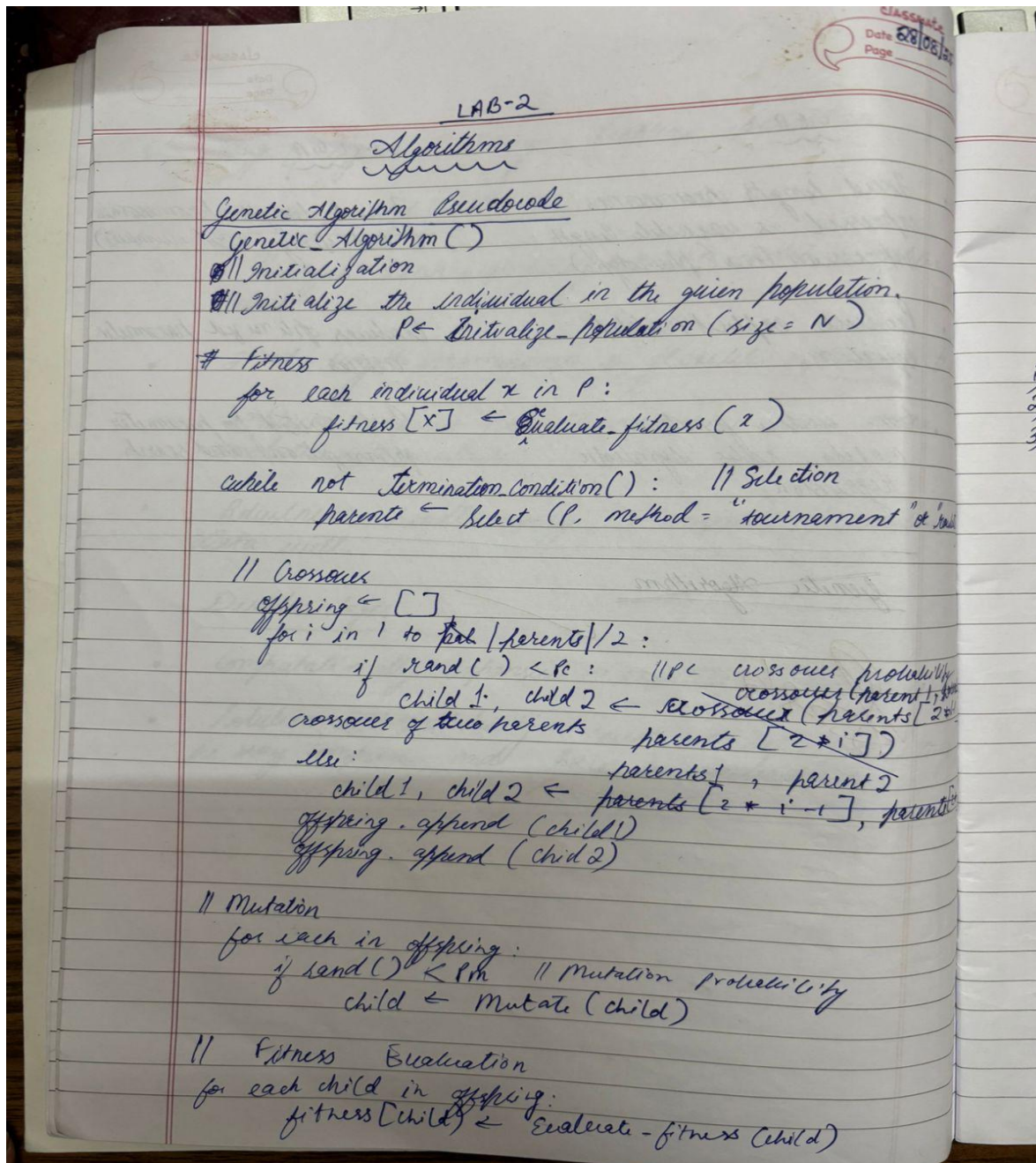
Program 1

Genetic Algorithm for Optimization Problems:

Genetic Algorithms (GA) are inspired by the process of natural selection and genetics, where the fittest individuals are selected for reproduction to produce the next generation. GAs are widely used for solving optimization and search problems. Implement a Genetic Algorithm using Python to solve a basic optimization problem, such as finding the maximum value of a mathematical function.

Application given: Robotic Process Implementation

Algorithm:



// Termination.

$P \leftarrow$ selection of survivor (P , offspring)
 $best \leftarrow$ fitness $f(x)$ over $x \in P$
 return $best$.

- Exerpts WRT - Robotic process.

- Generic version of genetic algorithm
- 1) Randomly initialize population P
 - 2) Determine fitness of population
 - 3) Until convergence repeat:
 - a) Select Parents from population
 - b) Perform mutation on new population
 - c) Crossover and generate new population
 - d) Calculate fitness for new population.

Genetic Alg

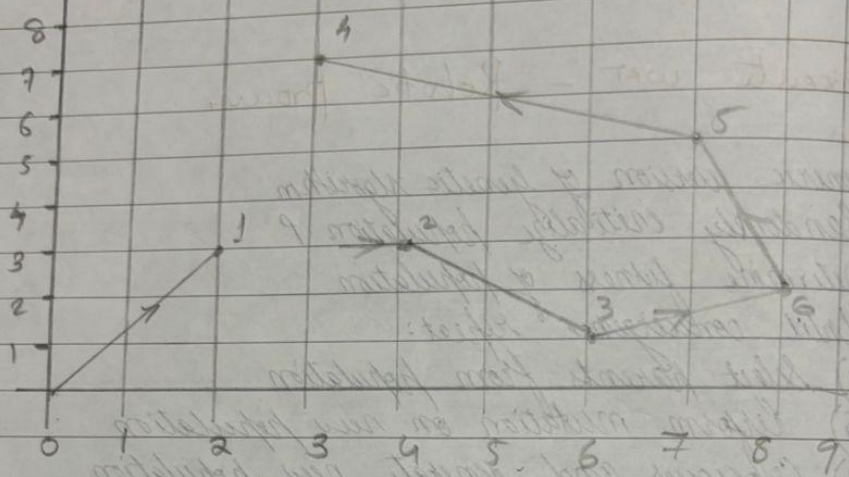
Robotic Process implementation:

Problem: Find the shortest path for a robot that visits all given waypoints starting from $(0,0)$ and returns the best possible sequence of waypoints.

Input: $[(0,0), (2,3), (4,3), (6,1), (3,7), (9,5), (8,2)]$

P.T.O

Graph reference:-



Output:

- 0) Generation 0 : Best distance = 23.7
- 1) Generation 10 : Best distance = 16.3
- 2) Generation 20 : Best distance = 16.3
- 3) Generation 30 : Best distance = 19.2
- 4) Generation 40 : Best distance = 20.1
- 5) Generation 50 : Best distance = 16.4
- 6) Generation 60 : Best distance = 16.3
- 7) Generation 70 : Best distance = 16.3
- 8) Generation 80 : Best distance = 16.3
- 9) Generation 90 : Best distance = 16.3
- 10) Generation 100 : Best distance = 16.3

Best path : [0, 1, 2, 3, 6, 5, 4]

Shortest distance path : 16.304457992

Code:

Robotic Process + Genetic Algorithm

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation, patches
from IPython.display import HTML
import random

# -----
# Robot & environment settings
# -----
DT = 0.2          # timestep for each command (seconds)
N_COMMANDS = 30   # number of sequential commands in a
                  # chromosome
WORLD_BOUNDS = (-5, 5, -5, 5) # x_min, x_max, y_min, y_max

start = np.array([-4.0, -4.0, 0.0]) # x, y, theta
goal = np.array([4.0, 4.0])         # goal position (x, y)
goal_radius = 0.2                   # considered reached if within this
radius

# command bounds
V_MIN, V_MAX = -1.0, 1.0 # linear velocity bounds (m/s)
W_MIN, W_MAX = -2.0, 2.0 # angular velocity bounds (rad/s)

# -----
# GA hyperparameters
# -----
```

```

POP_SIZE = 150
GENERATIONS = 120
TOURNAMENT_SIZE = 3
ELITISM = 2
CROSSOVER_RATE = 0.9
MUTATION_RATE = 0.12
MUTATION_STD_V = 0.2
MUTATION_STD_W = 0.4
RANDOM_SEED = 42

```

```

np.random.seed(RANDOM_SEED)
random.seed(RANDOM_SEED)

```

```

# -----
# Chromosome representation:
# sequence of (v, w) for N_COMMANDS
# -----
def create_random_chromosome():
    # shape (N_COMMANDS, 2)
    v = np.random.uniform(V_MIN, V_MAX,
size=N_COMMANDS)
    w = np.random.uniform(W_MIN, W_MAX,
size=N_COMMANDS)
    return np.vstack([v, w]).T

# -----
# Robot simulation (kinematic)
# -----
def simulate(chromosome, start_state=start, dt=DT,
record_poses=False):
    """

```

Simulate the robot executing the sequence of (v, w) commands.
Returns final state and optionally the full pose trajectory.

state: [x, y, theta]

"""

```
x, y, th = start_state.copy()
```

```
poses = [(x, y, th)]
```

```
for (v, w) in chromosome:
```

```
    # simple forward Euler integration of unicycle model
```

```
    x += v * np.cos(th) * dt
```

```
    y += v * np.sin(th) * dt
```

```
    th += w * dt
```

```
    # normalize theta to [-pi, pi]
```

```
    th = (th + np.pi) % (2*np.pi) - np.pi
```

```
    poses.append((x, y, th))
```

```
if record_poses:
```

```
    return np.array(poses)
```

```
else:
```

```
    return np.array([x, y, th])
```

```
# -----
```

```
# Fitness function
```

```
# -----
```

```
def fitness(chromosome, start_state=start):
```

```
    """
```

Higher fitness is better. We use negative final distance to goal plus bonuses.

We also reward reaching the goal early (shorter time).

```
    """
```

```
    poses = simulate(chromosome, start_state=start_state,  
record_poses=True)
```

```
    final_pos = poses[-1, :2]
```

```

dist = np.linalg.norm(final_pos - goal)
# base fitness: inverse of distance
fitness_value = -dist
# bonus if end close to goal
if dist <= goal_radius:
    # reward more if reached earlier (index is time step)
    # find earliest time within goal radius
    dists = np.linalg.norm(poses[:, :2] - goal.reshape(1,2), axis=1)
    idx = np.argwhere(dists <= goal_radius)
    if idx.size > 0:
        t_reached = idx[0, 0]
        # bigger reward for earlier reach
        fitness_value += 3.0 + 1.0 * (N_COMMANDS - t_reached)
/ N_COMMANDS
# penalty for leaving world bounds
x, y = poses[-1,0], poses[-1,1]
x_min, x_max, y_min, y_max = WORLD_BOUNDS
if not (x_min <= x <= x_max and y_min <= y <= y_max):
    fitness_value -= 5.0
return fitness_value

# -----
# GA operators
# -----
def tournament_selection(pop, fitnesses,
k=TOURNAMENT_SIZE):
    idxs = np.random.randint(0, len(pop), k)
    best_idx = idxs[0]
    for i in idxs:
        if fitnesses[i] > fitnesses[best_idx]:
            best_idx = i

```

```

    return pop[best_idx].copy()

def uniform_crossover(a, b):
    """Uniform crossover on each gene (v,w pair)"""
    child = a.copy()
    mask = np.random.rand(N_COMMANDS) < 0.5
    child[mask] = b[mask]
    return child

def mutate(chromosome, rate=MUTATION_RATE):
    for i in range(len(chromosome)):
        if np.random.rand() < rate:
            # mutate both v and w with gaussian noise and clamp
            chromosome[i,0] += np.random.normal(0,
MUTATION_STD_V)
            chromosome[i,1] += np.random.normal(0,
MUTATION_STD_W)
            chromosome[i,0] = np.clip(chromosome[i,0], V_MIN,
V_MAX)
            chromosome[i,1] = np.clip(chromosome[i,1], W_MIN,
W_MAX)
    return chromosome

# -----
# Initialize population
# -----
population = [create_random_chromosome() for _ in
range(POP_SIZE)]
best_history = []
avg_history = []

```

```

# -----
# Evolution loop
# -----
for gen in range(GENERATIONS):
    fitnesses = np.array([fitness(ind) for ind in population])
    sorted_idx = np.argsort(-fitnesses) # descending
    best_fitness = fitnesses[sorted_idx[0]]
    avg_fitness = fitnesses.mean()
    best_history.append(best_fitness)
    avg_history.append(avg_fitness)

    # print progress every N gens
    if gen % 10 == 0 or gen == GENERATIONS-1:
        print(f"Gen {gen:3d} | Best fitness: {best_fitness:.4f} | Avg:
{avg_fitness:.4f}")

    # create next population with elitism
    next_pop = []
    # Elitism: copy top ELITISM individuals
    for i in range(ELITISM):
        next_pop.append(population[sorted_idx[i]].copy())

    # fill the rest
    while len(next_pop) < POP_SIZE:
        parent1 = tournament_selection(population, fitnesses)
        parent2 = tournament_selection(population, fitnesses)
        # crossover
        if np.random.rand() < CROSSOVER_RATE:
            child = uniform_crossover(parent1, parent2)
        else:
            child = parent1.copy()

```

```

        # mutation
        child = mutate(child)
        next_pop.append(child)
    population = next_pop

# After evolution: get best individual
fitnesses = np.array([fitness(ind) for ind in population])
best_idx = np.argmax(fitnesses)
best_ind = population[best_idx]
print("\nFinished. Best final fitness:", fitnesses[best_idx])

# -----
# Visualize results
# -----
best_poses = simulate(best_ind, start_state=start,
                      record_poses=True)

# Plot fitness evolution
plt.figure(figsize=(9,4))
plt.plot(best_history, label="best")
plt.plot(avg_history, label="avg")
plt.xlabel("Generation")
plt.ylabel("Fitness")
plt.title("GA fitness over generations")
plt.legend()
plt.grid(True)
plt.show()

# Static plot of the best trajectory
fig, ax = plt.subplots(figsize=(6,6))
x_min, x_max, y_min, y_max = WORLD_BOUNDS

```

```

ax.set_xlim(x_min-0.5, x_max+0.5)
ax.set_ylim(y_min-0.5, y_max+0.5)
ax.set_aspect('equal')

# Draw start and goal
ax.plot(start[0], start[1], 'go', label='start')
ax.plot(goal[0], goal[1], 'r*', markersize=14, label='goal')
ax.add_patch(plt.Circle((goal[0], goal[1]), goal_radius, color='red',
alpha=0.15))

# Draw trajectory
traj_xy = best_poses[:, :2]
ax.plot(traj_xy[:,0], traj_xy[:,1], '-o', markersize=3, linewidth=1,
label='best trajectory')

ax.set_title('Best trajectory (final)')
ax.legend()
plt.grid(True)
plt.show()

# -----
# Animation (inline in colab)
# -----
fig2, ax2 = plt.subplots(figsize=(6,6))
ax2.set_xlim(x_min-0.5, x_max+0.5)
ax2.set_ylim(y_min-0.5, y_max+0.5)
ax2.set_aspect('equal')

ax2.plot(start[0], start[1], 'go', label='start')
ax2.plot(goal[0], goal[1], 'r*', markersize=14, label='goal')
ax2.add_patch(plt.Circle((goal[0], goal[1]), goal_radius, color='red',

```

```
alpha=0.15))
```

```
line, = ax2.plot([], [], '-o', markersize=5)
robot_patch = patches.Circle((start[0], start[1]), 0.12, fc='blue')
ax2.add_patch(robot_patch)
```

```
def init_anim():
    line.set_data([], [])
    robot_patch.center = (start[0], start[1])
    return line, robot_patch
```

```
def animate(i):
    if i >= len(best_poses):
        i = len(best_poses)-1
    xs = best_poses[:i+1, 0]
    ys = best_poses[:i+1, 1]
    line.set_data(xs, ys)
    robot_patch.center = (best_poses[i,0], best_poses[i,1])
    return line, robot_patch
```

```
anim = animation.FuncAnimation(fig2, animate,
init_func=init_anim,
frames=len(best_poses), interval=120,
blit=True)
```

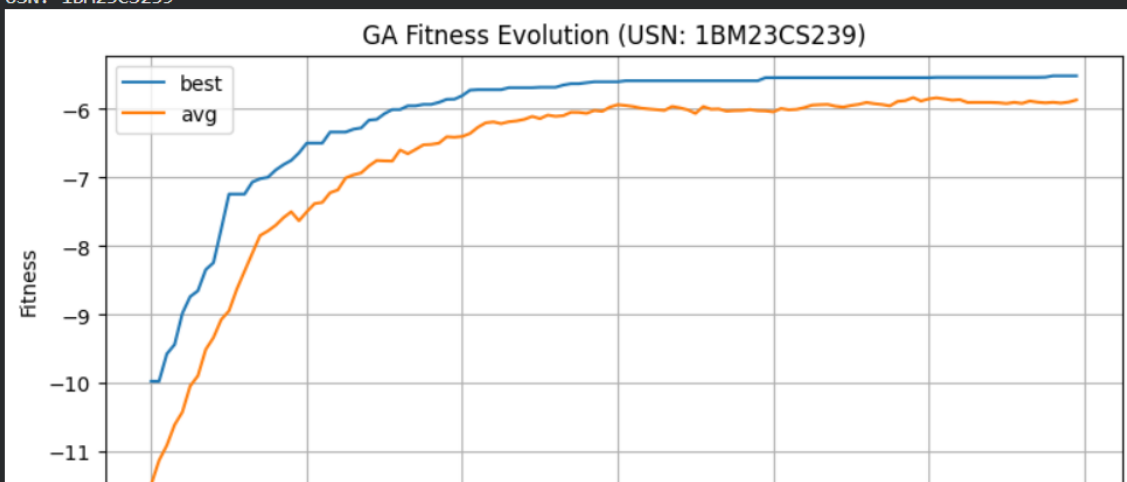
```
# Display animation in notebook
HTML(anim.to_jshtml())
```

```
Gen 0 | Best: -9.9701 | Avg: -11.4759
Gen 10 | Best: -7.2406 | Avg: -8.9436
.. Gen 20 | Best: -6.4974 | Avg: -7.4991
Gen 30 | Best: -6.0678 | Avg: -6.7565
Gen 40 | Best: -5.8059 | Avg: -6.4007
Gen 50 | Best: -5.6834 | Avg: -6.1407
Gen 60 | Best: -5.6031 | Avg: -5.9375
Gen 70 | Best: -5.5885 | Avg: -6.0651
Gen 80 | Best: -5.5433 | Avg: -6.0421
Gen 90 | Best: -5.5433 | Avg: -5.9488
Gen 100 | Best: -5.5433 | Avg: -5.8527
Gen 110 | Best: -5.5386 | Avg: -5.9201
Gen 119 | Best: -5.5165 | Avg: -5.8682
```

Finished Evolution

Best fitness: -5.5164994597917865

USN: 1BM23CS239



Program 2

Particle Swarm Optimization for Function Optimization:

Particle Swarm Optimization (PSO) is inspired by the social behavior of birds flocking or fish schooling. PSO is used to find optimal solutions by iteratively improving a candidate solution with regard to a given measure of quality. Implement the PSO algorithm using Python to optimize a mathematical function.

Application given : Image processing

Algorithm:

Lab-4

PARTICLE SWARM OPTIMIZATION

Optimization algorithm inspired by social behaviour of birds flocking or fish schooling.

Algorithm

1. Initialize the swarm:
Each particle has its position, velocity and remembers its pbest.

import random

def objective(x):
 # according to requirement

particles = []

for i in range(num_particles)

pos = [random.uniform(-10, 10)]

vel = [random.uniform(-1, 1)]

particles.append({

"position": pos,

"velocity": vel,

"pbest": pos[:]})

gbest = min(particles, key=lambda p:
 objective(p["pbest"]))

2. Evaluate fitness:

update pbest and gbest

```

for p in particles:
    if objective(p["position"]) < objective(p["pbest"]):
        p["pbest"] = p["position"][:]

```

```

gbest = min(particles, key=lambda p:
            objective(p["pbest"]))["pbest"][:]

```

3. Update velocity and position

```

for p in particles:
    new_velocity = []
    new_position = []

```

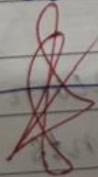
```

for d in range(dimensions):
    new_vel.append(vel[d])
    new_pos.append(pos[d])

```

4. Run loop

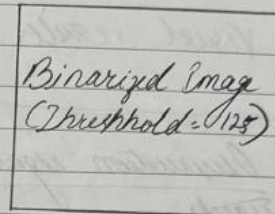
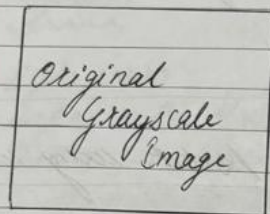
The same iteration is done for specific number of times or until minimum convergence.



PSO
↓
Img. Processing

PSO in Image Processing

Output:-



Iteration 10/50, best Otsu variance: 1423.8369
Iteration 20/50, best Otsu variance: 1423.8369
Iteration 30/50, best Otsu variance: 1423.8369
Iteration 40/50, best Otsu variance: 1423.8369
Iteration 50/50, best Otsu variance: 1423.8369

Threshold for all: 125

Optimal Threshold found by PSO: 125

Observation: The PSO algorithm successfully determined an optimal threshold value for binarizing the grayscale image. By maximizing Otsu's between-class variance, PSO identified a threshold that effectively separates the foreground from the background in the image.

Output graphs:

* Convergence over iterations - The PSO improved threshold value, indicated by increasing between class variance and stabilization of the best threshold.

Code:

```
# Particle Swarm Optimization for Image Processing  
# USN Display: 1BM23CS239
```

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from google.colab import files
```

```
USN = "1BM23CS239" # <<< YOUR USN HERE
```

```
# -----
```

```
# 1. Upload Image
```

```
# -----
```

```
print("Upload an image (any format).")  
uploaded = files.upload()
```

```
for fn in uploaded.keys():  
    input_image_path = fn
```

```
# Read image
```

```
img = cv2.imread(input_image_path)  
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# -----
```

```
# 2. PSO Parameters
```

```
# -----
```

```

num_particles = 30
max_iter = 50
w = 0.7    # inertia
c1 = 1.5   # cognitive
c2 = 1.5   # social

# -----
# 3. Fitness Function (Otsu-inspired)
# -----
def fitness(threshold):
    threshold = int(threshold)
    _, binary = cv2.threshold(gray, threshold, 255,
cv2.THRESH_BINARY)

    # Otsu-like variance metric (higher is better)
    pixels = gray.flatten()

    class0 = pixels[pixels <= threshold]
    class1 = pixels[pixels > threshold]

    if len(class0) == 0 or len(class1) == 0:
        return 0

    w0 = len(class0) / len(pixels)
    w1 = len(class1) / len(pixels)

    m0 = np.mean(class0)
    m1 = np.mean(class1)

    return w0 * w1 * ((m0 - m1)**2)

```

```

# -----
# 4. Initialize PSO
# -----
particles = np.random.uniform(0, 255, num_particles)      #
thresholds
velocity = np.zeros(num_particles)

pbest = particles.copy()
pbest_fitness = np.array([fitness(t) for t in particles])

gbest = pbest[np.argmax(pbest_fitness)]
gbest_fitness = max(pbest_fitness)


# -----
# 5. PSO Loop
# -----
fitness_history = []

for i in range(max_iter):
    for j in range(num_particles):

        # Update velocity
        r1, r2 = np.random.rand(), np.random.rand()
        velocity[j] = (w * velocity[j] +
                       c1 * r1 * (pbest[j] - particles[j]) +
                       c2 * r2 * (gbest - particles[j]))

        # Update particle

```

```

particles[j] += velocity[j]

# Clamp threshold
particles[j] = np.clip(particles[j], 0, 255)

# Fitness
f = fitness(particles[j])

# Update personal best
if f > pbest_fitness[j]:
    pbest[j] = particles[j]
    pbest_fitness[j] = f

# Update global best
if f > gbest_fitness:
    gbest = particles[j]
    gbest_fitness = f

fitness_history.append(gbest_fitness)
print(f'Iteration {i+1}/{max_iter}, Best Threshold = {int(gbest)},
Fitness = {gbest_fitness:.4f}')

best_threshold = int(gbest)
print("\nPSO Complete!")
print("Best Threshold:", best_threshold)
print("USN:", USN)

# -----
# 6. Final Binarized Output
# -----
_, binary_final = cv2.threshold(gray, best_threshold, 255,

```

```
cv2.THRESH_BINARY)
```

```
# -----
```

```
# 7. Display Results
```

```
# -----
```

```
plt.figure(figsize=(15,6))
```

```
plt.subplot(1,3,1)
```

```
plt.imshow(img_rgb)
```

```
plt.title(f'Original Image\nUSN: {USN}')
```

```
plt.axis("off")
```

```
plt.subplot(1,3,2)
```

```
plt.imshow(gray, cmap='gray')
```

```
plt.title("Grayscale Image")
```

```
plt.axis("off")
```

```
plt.subplot(1,3,3)
```

```
plt.imshow(binary_final, cmap='gray')
```

```
plt.title(f'Binarized Image (PSO Threshold={best_threshold})')
```

```
plt.axis("off")
```

```
plt.show()
```

```
# -----
```

```
# 8. Plot PSO Convergence
```

```
# -----
```

```
plt.figure(figsize=(8,4))
```

```
plt.plot(fitness_history, marker='o')
```

```
plt.title(f'PSO Fitness Convergence (USN: {USN})')
plt.xlabel("Iteration")
plt.ylabel("Fitness")
plt.grid(True)
plt.show()
```

output:

Upload an image (any format).

WhatsApp Image 2025-11-18 at 10.33.39 PM (2).jpeg

WhatsApp Image 2025-11-18 at 10.33.39 PM (2).jpeg(image/jpeg)

- 178516 bytes, last modified: 18/11/2025 - 100% done

Saving WhatsApp Image 2025-11-18 at 10.33.39 PM (2).jpeg to

WhatsApp Image 2025-11-18 at 10.33.39 PM (2).jpeg

Iteration 1/50, Best Threshold = 120, Fitness = 2187.4603

Iteration 2/50, Best Threshold = 119, Fitness = 2187.7561

Iteration 3/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 4/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 5/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 6/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 7/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 8/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 9/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 10/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 11/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 12/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 13/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 14/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 15/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 16/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 17/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 18/50, Best Threshold = 118, Fitness = 2187.8321

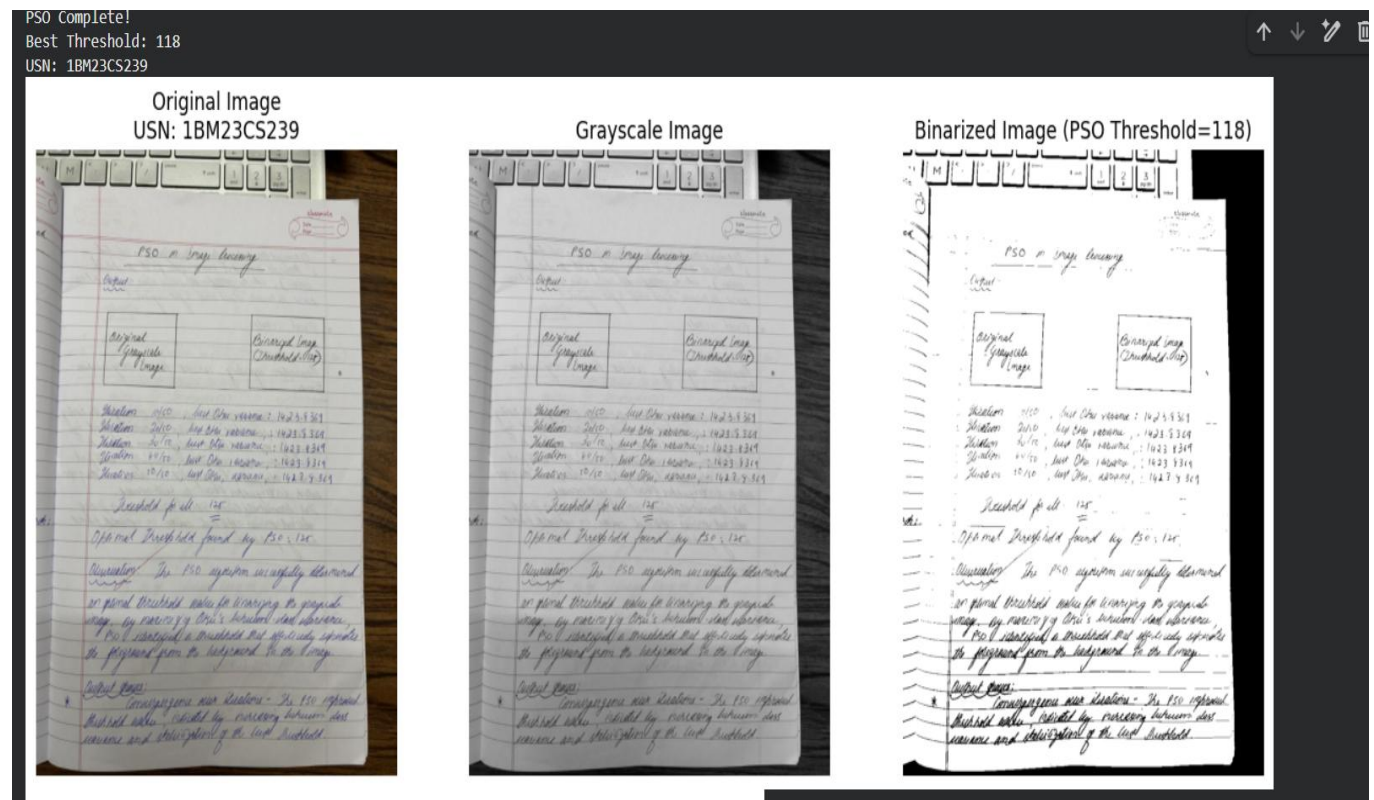
Iteration 19/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 20/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 21/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 22/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 23/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 24/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 25/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 26/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 27/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 28/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 29/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 30/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 31/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 32/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 33/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 34/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 35/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 36/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 37/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 38/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 39/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 40/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 41/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 42/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 43/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 44/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 45/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 46/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 47/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 48/50, Best Threshold = 118, Fitness = 2187.8321
Iteration 49/50, Best Threshold = 118, Fitness = 2187.8321

Iteration 50/50, Best Threshold = 118, Fitness = 2187.8321

PSO Complete!

Best Threshold: 118

USN: 1BM23CS239



Program 3:

Ant Colony Optimization for the Traveling Salesman Problem:

The foraging behaviour of ants has inspired the development of optimization algorithms that can solve complex problems such as the Traveling Salesman Problem (TSP). Ant Colony Optimization (ACO) simulates the way ants find the shortest path between food sources and their nest. Implement the ACO algorithm using Python to solve the TSP, where the objective is to find the shortest possible route that visits a list of cities and returns to the origin city.

Application given : Packets Transmission via Router
Algorithm:

General Algorithm for ACO

Initialize pheromone values. Let τ on all of the solution components. Set parameters α (pheromone influence), β (heuristic influence), evaporation rate ρ , no. of ants m .

while stopping criteria not met:

for each ant k in 1 to m :

initialize an empty solution s_k

while solution s_k is incomplete:

select next solution component c based on probability proportional to:

$$[\tau(c)]^\alpha * [\eta(c)]^\beta$$

where $\eta(c)$ is heuristic desirability of component c .

Add component c to solution s_k . Evaluate the quality of solution s_k .

for each solution component c :

evaporate pheromone:

$$\tau(c) = (1 - \rho) * \tau(c)$$

for each ant k

deposit pheromone on components in solution

$\Delta \tau$:

$$\tau(c) = \tau(c) + \Delta \tau = \tau(c) + \frac{1}{f(s_k)}$$

amount $\Delta \tau = \frac{1}{f(s_k)}$ depends on quality of the solution.

return best solution found.

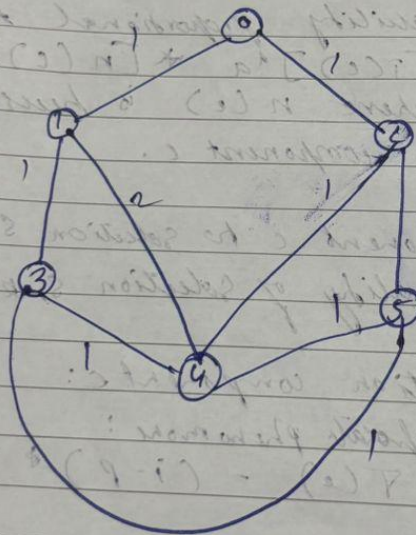
Packets
Via Router
Transmission

Packets Transmission via Router

Input: adjacency matrix

0	1	1	0	0	0
1	0	0	1	2	0
1	0	0	0	1	1
0	1	0	0	0	1
0	2	1	0	0	1
0	0	1	1	1	0

Graph is:



start node: 0

Destination node: 5

Output: 0 → 1 → 3 → 5

cost: 3

Goal: To implement a routing strategy using the Ant Colony Optimization algorithm that simulates how

Code:

```
# Ant Colony Optimization for Packet Routing
# Includes graph visualization + USN output
# USN: 1BM23CS239
```

```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
```

```
USN = "1BM23CS239"  # <<< YOUR USN HERE
```

```
# -----
```

```
# 1. Create Network Graph
```

```
# -----
```

```
G = nx.Graph()
```

```
edges = [
    ("A","B",4), ("A","C",3),
    ("B","D",2), ("C","D",4),
    ("C","E",2), ("D","E",3),
    ("D","F",2), ("E","F",3)
]
```

```
# Add weighted edges
```

```
for u,v,w in edges:
```

```
    G.add_edge(u, v, weight=w)
```

```
pos = nx.spring_layout(G, seed=42)  # Node layout
```

```

# -----
# 2. ACO Parameters
# -----
num_ants = 15
num_iterations = 40
alpha = 1      # pheromone importance
beta = 2       # heuristic importance (1/distance)
evaporation = 0.5
pheromone_constant = 100

nodes = list(G.nodes)
pheromone = np.ones((len(nodes), len(nodes)))

node_index = {node:i for i,node in enumerate(nodes)}
index_node = {i:node for node,i in node_index.items()}

# -----
# 3. Choose Source and Destination
# -----
start = "A"
end = "F"

print("Source:", start)
print("Destination:", end)
print("USN:", USN)

# -----
# 4. Helper: Path Distance
# -----

```

```
def path_distance(path):
    dist = 0
    for i in range(len(path)-1):
        dist += G[path[i]][path[i+1]]['weight']
    return dist
```

```
# -----
# 5. ACO Algorithm
# -----
```

```
best_path = None
best_cost = float("inf")
cost_history = []
```

```
for iteration in range(num_iterations):
    all_paths = []
    all_costs = []
```

```
# -----
# Each ant constructs a path
# -----
```

```
for ant in range(num_ants):
    path = [start]
    current = start
```

```
    visited = set([start])
```

```
    while current != end:
        neighbors = list(G.neighbors(current))
```

```
        # Probabilities based on pheromone & heuristic
```

```

probs = []
for nxt in neighbors:
    if nxt not in visited:
        i = node_index[current]
        j = node_index[nxt]
        tau = pheromone[i][j]
        eta = 1.0 / G[current][nxt]['weight']
        probs.append((nxt, (tau**alpha) * (eta**beta)))

if len(probs) == 0:
    break

nodes_list, prob_values = zip(*probs)
prob_values = np.array(prob_values)
prob_values = prob_values / prob_values.sum()

next_node = np.random.choice(nodes_list, p=prob_values)
path.append(next_node)
visited.add(next_node)
current = next_node

# Evaluate path
if path[-1] == end:
    cost = path_distance(path)
    all_paths.append(path)
    all_costs.append(cost)

if cost < best_cost:
    best_cost = cost
    best_path = path

```

```

# -----
# Pheromone Update
# -----
pheromone = (1 - evaporation) * pheromone # evaporation

for i, path in enumerate(all_paths):
    cost = all_costs[i]
    deposit = pheromone_constant / cost
    for k in range(len(path) - 1):
        a = node_index[path[k]]
        b = node_index[path[k + 1]]
        pheromone[a][b] += deposit
        pheromone[b][a] += deposit

    cost_history.append(best_cost)
    print(f'Iteration {iteration+1}/{num_iterations}, Best Cost =
{best_cost}, Best Path = {best_path}')

# -----
# 6. Final Best Route
# -----
print("\nBest Route Found:", best_path)
print("Best Path Cost:", best_cost)
print("USN:", USN)

# -----
# 7. Draw Network + Best Route
# -----
plt.figure(figsize=(10,6))

```

```
# Draw full network
nx.draw(G, pos, with_labels=True, node_color='skyblue',
node_size=1400, font_size=12)
labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=labels)
```

```
# Highlight best path
best_edges = [(best_path[i], best_path[i+1]) for i in
range(len(best_path)-1)]
nx.draw_networkx_edges(G, pos, edgelist=best_edges, width=4,
edge_color='red')
```

```
plt.title(f'ACO Packet Routing Graph\nBest Path: {best_path} |
USN: {USN}')
plt.show()
```

```
# -----
# 8. Plot Cost Convergence
# -----
plt.figure(figsize=(8,4))
plt.plot(cost_history, marker='o')
plt.xlabel("Iteration")
plt.ylabel("Path Cost")
plt.grid(True)
plt.title(f'ACO Cost Convergence (USN: {USN})')
plt.show()
```

```
output:
Source: A
Destination: F
USN: 1BM23CS239
```

Iteration 1/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 2/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 3/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 4/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 5/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 6/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 7/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 8/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 9/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 10/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 11/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 12/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 13/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 14/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 15/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 16/40, Best Cost = 8, Best Path = ['A', np.str_('B'),

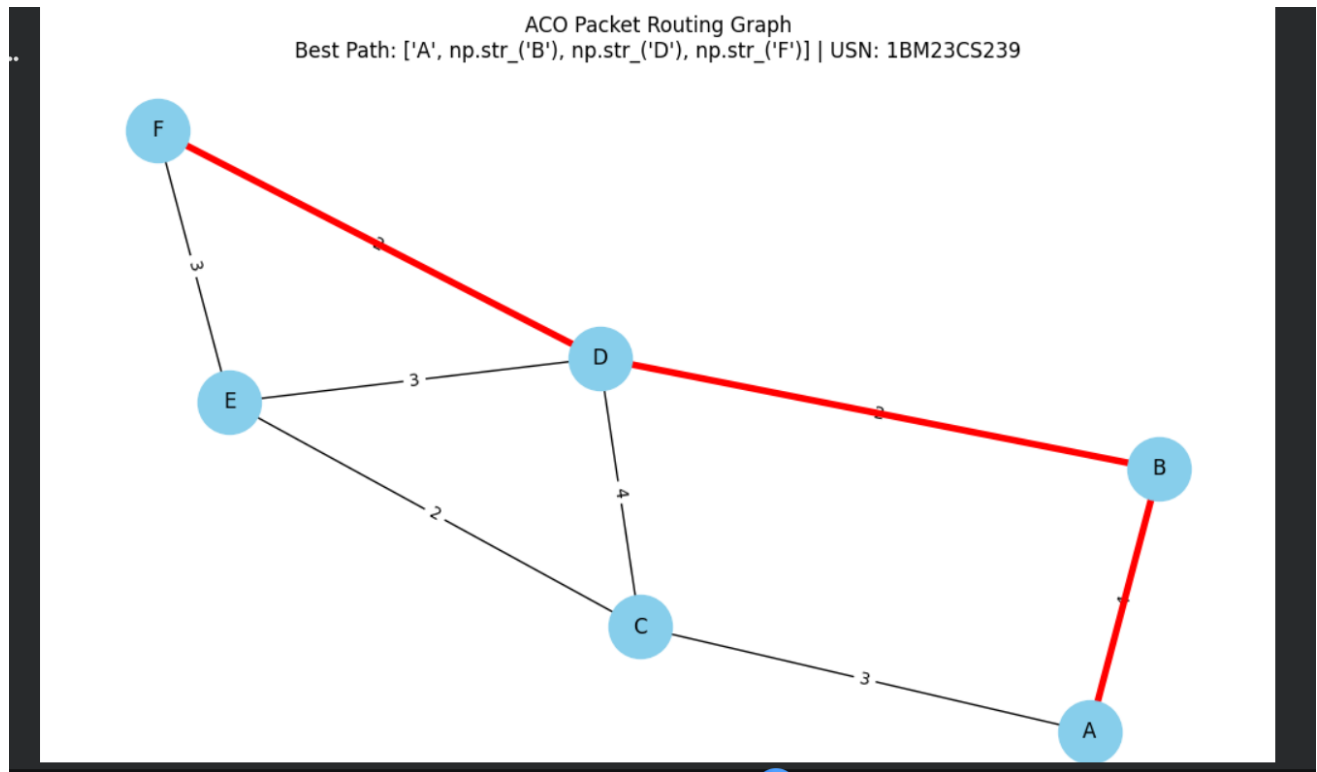
np.str_('D'), np.str_('F')]
 Iteration 17/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 18/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 19/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 20/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 21/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 22/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 23/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 24/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 25/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 26/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 27/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 28/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 29/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 30/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]
 Iteration 31/40, Best Cost = 8, Best Path = ['A', np.str_('B'),
 np.str_('D'), np.str_('F')]

Iteration 32/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 33/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 34/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 35/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 36/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 37/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 38/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 39/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]
Iteration 40/40, Best Cost = 8, Best Path = ['A', np.str_('B'), np.str_('D'), np.str_('F')]

Best Route Found: ['A', np.str_('B'), np.str_('D'), np.str_('F')]

Best Path Cost: 8

USN: 1BM23CS239



Program 4

Cuckoo Search (CS):

Cuckoo Search (CS) is a nature-inspired optimization algorithm based on the brood parasitism of some cuckoo species. This behaviour involves laying eggs in the nests of other birds, leading to the optimization of survival strategies. CS uses Lévy flights to generate new solutions, promoting global search capabilities and avoiding local minima. The algorithm is widely used for solving continuous optimization problems and has applications in various domains, including engineering design, machine learning, and data

mining.

Application given: Optimization in Power Consumption
Algorithm:

Date 24/10/25
Page

Ant Colony Search Algorithm

Algorithm

AntColonySearch()

Initialize population of n best nests (solutions)
 $x_i, (i = 1, 2, \dots, n)$

Define the objective function $f(x), x = (x_1, x_2, \dots, x_d)^T$

Find the current best solution x_{best} - best among the nests

while (termination or stopping criteria is not met) do:

for each ant $k (k = 1 \text{ to } n)$ do

Generate a new solution x_i^{new} by using flight

from x_i

Evaluate fitness $f(x_i^{\text{new}})$

Randomly choose a nest j among n

if $(f(x_i^{\text{new}}) < f(x_j))$ then

replace x_j with x_i^{new}

end if

end for

A fraction (p_a) of worse nests are abandoned and replaced with new random solutions.

Keep the best solution (x_{best}) among current nests.

Rank the nests and find the new x_{best}

$t = t + 1$

end while

Return x_{best} as the optimal solution found.

Optimizing in n/w packets

Problem Statement for Cuckoo SearchOptimization $\rightarrow$ Power Consumption

Output:

Iteration	1/100	-	Best Power Consumption:	21.488
Iteration	10/100	-	Best Power Consumption:	20.7345
Iteration	20/100	-	Best Power Consumption:	20.3148
Iteration	30/100	-	Best Power Consumption:	19.7958
Iteration	40/100	-	Best Power Consumption:	19.6848
Iteration	50/100	-	Best Power Consumption:	19.7163
Iteration	60/100	-	Best Power Consumption:	19.7163
Iteration	70/100	-	Best Power Consumption:	19.7163
Iteration	80/100	-	Best Power Consumption:	19.7163
Iteration	90/100	-	Best Power Consumption:	19.6125
Iteration	100/100	-	Best Power Consumption:	19.6125

Optimization Results

Optimal Parameters : [4.5885625 0.57364008 1.39597217]

Minimum Power Consumption: 19.6125039

Code:

```
# -----  
# Optimization in Power Consumption using  
# Cuckoo Search Algorithm (CSA)  
  
# Includes graph and prints USN: 1BM23CS239  
# -----  
  
import numpy as np  
import matplotlib.pyplot as plt  
import math # Import the math module  
  
# Your USN  
USN = "1BM23CS239"  
print("Running Cuckoo Search Optimization")  
print("USN:", USN)  
print("-----")  
  
# -----  
# Define the Power Consumption Model (example)  
#  $P(x) = a \cdot x^2 + b \cdot x + c$  (x = operating load percentage)  
# -----  
def power_consumption(x):  
    a, b, c = 0.02, -1.5, 200 # sample model  
    return a*(x**2) + b*x + c  
  
# -----  
# Cuckoo Search Algorithm Implementation  
# -----  
def cuckoo_search(n=20, iterations=200, pa=0.25, lb=0, ub=100):
```

```

# Generate initial population
population = np.random.uniform(lb, ub, n)
fitness = np.array([power_consumption(x) for x in population])

best_index = np.argmin(fitness)
best = population[best_index]
best_fitness = fitness[best_index]

history = []

for t in range(iterations):
    # Levy flight
    beta = 1.5
    sigma = ( math.gamma(1+beta) * np.sin(np.pi*beta/2) \
              / ( math.gamma((1+beta)/2) * beta*2**((beta-1)/2) )
    )**(1/beta)

    levy = np.random.normal(0, sigma, n) /
    np.abs(np.random.normal(0, 1, n))**(1/beta)

    new_population = population + levy

    # Bounds check
    new_population = np.clip(new_population, lb, ub)

    new_fitness = np.array([power_consumption(x) for x in
    new_population])

    # Greedy replacement
    for i in range(n):
        if new_fitness[i] < fitness[i]:

```

```

        population[i] = new_population[i]
        fitness[i] = new_fitness[i]

    # Abandon nests
    abandon = np.random.rand(n) < pa
    random_new = np.random.uniform(lb, ub, sum(abandon))
    population[abandon] = random_new
    fitness[abandon] = np.array([power_consumption(x) for x in
random_new])

    # Track best
    best_index = np.argmin(fitness)
    best = population[best_index]
    best_fitness = fitness[best_index]

    history.append(best_fitness)

return best, best_fitness, history

# -----
# Run the Optimization
# -----
best_x, best_cost, history = cuckoo_search()

print("Optimal Operating Load (x):", best_x)
print("Minimum Power Consumption Cost:", best_cost)

# -----
# Plotting Results
# -----

```

```

# 1. Fitness graph
plt.figure(figsize=(8,5))
plt.plot(history)
plt.title("Cuckoo Search Optimization – Power Consumption")
plt.xlabel("Iteration")
plt.ylabel("Best Power Cost")
plt.grid(True)
plt.show()

# 2. Plot original power function & optimized point
x_vals = np.linspace(0,100,400)
y_vals = power_consumption(x_vals)

plt.figure(figsize=(8,5))
plt.plot(x_vals, y_vals, label="Power Consumption Curve")
plt.scatter([best_x], [best_cost], color='red', s=80, label="Optimized
Point")
plt.title("Power Consumption vs Load (%)")
plt.xlabel("Load (%)")
plt.ylabel("Power Cost")
plt.legend()
plt.grid(True)
plt.show()

print("\nOptimization Completed Successfully for USN:", USN)

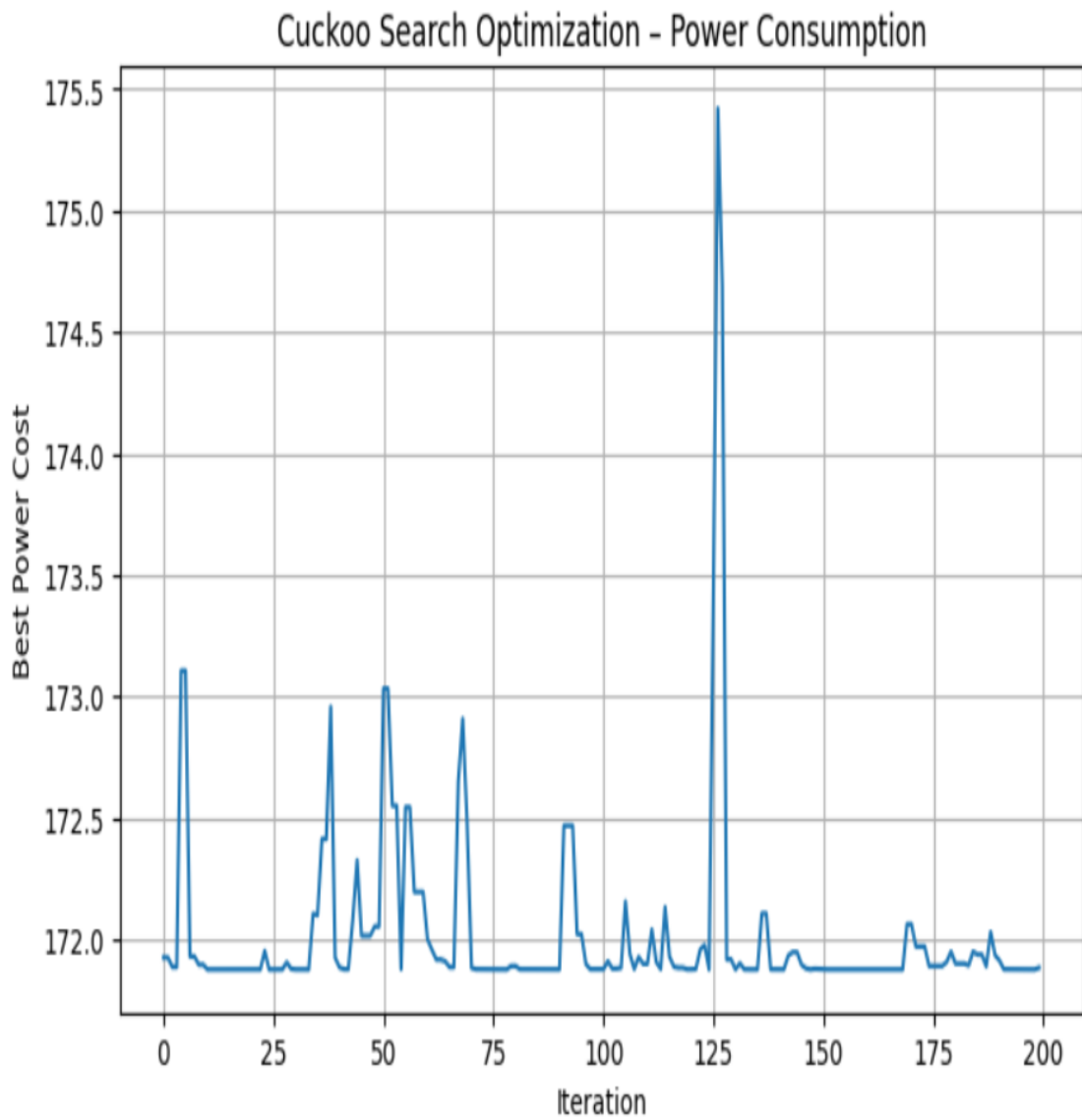
```

Running Cuckoo Search Optimization

USN: 1BM23CS239

Optimal Operating Load (x): 36.908042778976544

Minimum Power Consumption Cost: 171.88200826703044



Power Consumption vs Load (%)

Program 5

Grey Wolf Optimizer (GWO):

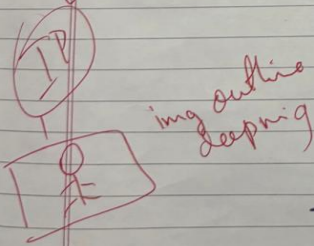
The Grey Wolf Optimizer (GWO) algorithm is a swarm intelligence algorithm inspired by the social hierarchy and hunting behavior of grey wolves. It mimics the leadership structure of alpha, beta, delta, and omega wolves and their collaborative hunting strategies. The GWO algorithm uses these social hierarchies to model the optimization process, where the alpha wolves guide the search process while beta and delta wolves assist in refining the search direction. This algorithm is effective for continuous optimization problems and has applications in engineering, data analysis, and machine learning.

Application given: Image Outline Deepening
Algorithm:

Grey Wolf Search Optimization

Algorithm

1. Begin GWO
2. Define the objective function $f(x)$, $x = (x_1, x_2, \dots, x_n)$
3. Initialize parameters:
 - Number of Wolves (N)
 - Number of Dimensions (D)
 - Number of maximum iterations (Max-Iters)
 - Search space bounds (lb, ub)
4. Initialize the population of grey wolves
 x_i ($i = 1, 2, \dots, N$)
 - For each wolf i :
 - For each dimension j :
 - $x_{ij} = \text{random}(lb, ub)$
5. Evaluate the fitness of every wolf
 $f(x_i) = f(x_i)$
6. Identify the top 3 wolves based on fitness:
 - alpha = best (lowest fitness)
 - beta = second best
 - delta = third best
7. For each iteration $t = 1$ to Max-Iters do:
 - Update the control parameter
 $a = t - t^* (t / \text{max-Iters})$
 - For each wolf $i = 1$ to N do:



- For each dimension $j=1$ to D do:
 Generate random numbers $r_1, r_2 \in [0,1]$
 Compute coefficients:

$$A1 = 2 * a * r_1 - a$$

$$C1 = 2 * a * r_2$$

Compute the distance to Alpha:

$$D_{\alpha} = [C1 * \alpha[j] - x_i[j]]$$

$$x_1 = \alpha[j] - A1 * D_{\alpha}$$

Repeat for Beta and Delta:

$$(x_2, C2, D_{\beta}, x_2)$$

$$(x_3, C3, D_{\delta}, x_3)$$

Update position:

$$x_i[j] = (x_1 + x_2 + x_3) / 3$$

8. Record best fitness

9. End for

10. Return Alpha and $f(\alpha)$ (Best fitness)

Date _____
Page _____

Image Outline Detection

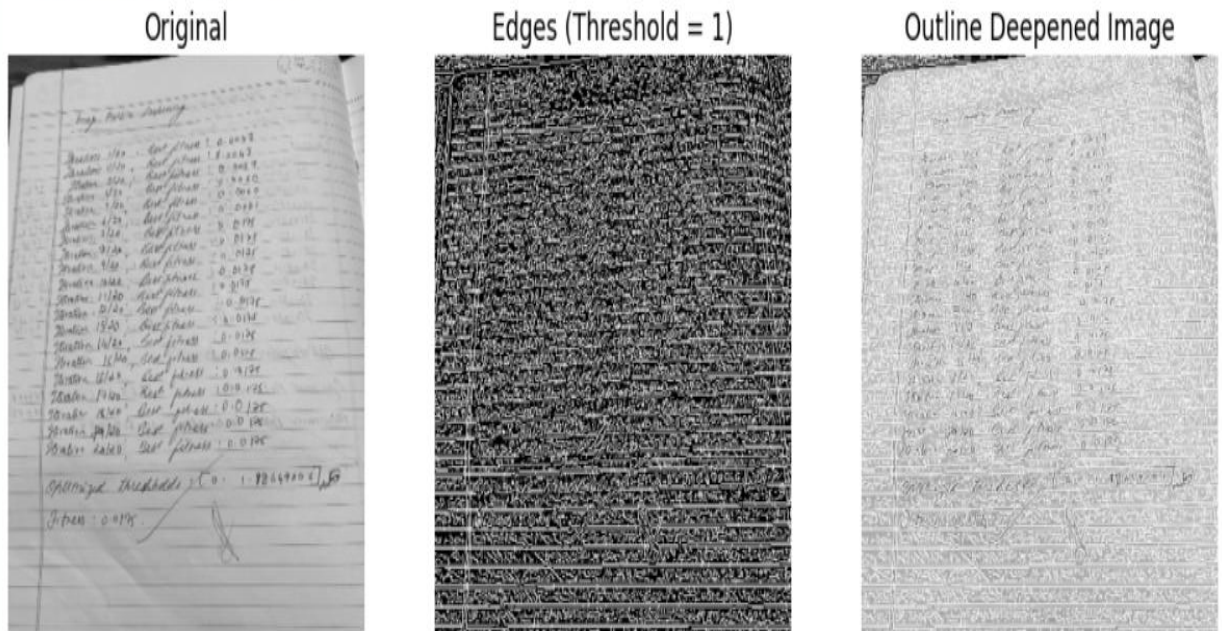
Iteration 1/20	Best fitness : 0.0047
Iteration 2/20	Best fitness : 0.0047
Iteration 3/20	Best fitness : 0.0059
Iteration 4/20	Best fitness : 0.0060
Iteration 5/20	Best fitness : 0.0060
Iteration 6/20	Best fitness : 0.0081
Iteration 7/20	Best fitness : 0.0175
Iteration 8/20	Best fitness : 0.0175
Iteration 9/20	Best fitness : 0.0175
Iteration 10/20	Best fitness : 0.0175
Iteration 11/20	Best fitness : 0.0175
Iteration 12/20	Best fitness : 0.0175
Iteration 13/20	Best fitness : 0.0175
Iteration 14/20	Best fitness : 0.0175
Iteration 15/20	Best fitness : 0.0175
Iteration 16/20	Best fitness : 0.0175
Iteration 17/20	Best fitness : 0.0175
Iteration 18/20	Best fitness : 0.0175
Iteration 19/20	Best fitness : 0.0175
Iteration 20/20	Best fitness : 0.0175

Optimized thresholds : [0.188649006]

Fitness : 0.0175

Code:

```
/tmp/ipython-input-1918535857.py:37: RuntimeWarning: overflow encountered in scalar negative  
return -np.sum(edges) # maximize edges → minimize negative edges  
Best Threshold (GWO optimized): 1
```



```
# -----  
# Image Outline Deepening using Grey Wolf Optimization (GWO)  
# Google Colab Code  
# Includes your USN: 1BM23CS239  
# -----
```

```
import cv2  
import numpy as np  
import matplotlib.pyplot as plt
```

```
USN = "1BM23CS239"  
print("Running GWO-Based Image Outline Deepening")  
print("USN:", USN)  
print("-----")
```

```

# -----
# Upload an image
# -----
from google.colab import files
uploaded = files.upload()

# Load the first uploaded image
img_path = list(uploaded.keys())[0]
img = cv2.imread(img_path, 0) # grayscale
img = cv2.resize(img, (400, 400)) # normalize size
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.show()

# -----
# Fitness Function (edge strength)
# -----
def fitness(threshold):
    threshold = int(threshold)
    edges = cv2.Canny(img, threshold, threshold * 2)
    return -np.sum(edges) # maximize edges → minimize negative
edges

# -----
# Grey Wolf Optimization (GWO)
# -----
def GWO(max_iter=50, wolves=15, lb=1, ub=255):
    alpha, beta, delta = None, None, None
    a_values = []

```

```
population = np.random.uniform(lb, ub, wolves)
fitness_values = np.array([fitness(x) for x in population])
```

```
for t in range(max_iter):
```

```
    # Sort by fitness (ascending, because lower = better)
```

```
    sorted_idx = np.argsort(fitness_values)
```

```
    alpha = population[sorted_idx[0]]
```

```
    beta = population[sorted_idx[1]]
```

```
    delta = population[sorted_idx[2]]
```

```
    a = 2 - (2 * t) / max_iter
```

```
    a_values.append(fitness_values[sorted_idx[0]])
```

```
    new_population = []
```

```
    for i in range(wolves):
```

```
        r1, r2 = np.random.random(), np.random.random()
```

```
        A1 = 2*a*r1 - a
```

```
        C1 = 2*r2
```

```
        D_alpha = abs(C1 * alpha - population[i])
```

```
        X1 = alpha - A1 * D_alpha
```

```
        r1, r2 = np.random.random(), np.random.random()
```

```
        A2 = 2*a*r1 - a
```

```
        C2 = 2*r2
```

```
        D_beta = abs(C2 * beta - population[i])
```

```
        X2 = beta - A2 * D_beta
```

```
        r1, r2 = np.random.random(), np.random.random()
```

```
A3 = 2*a*r1 - a
C3 = 2*r2
```

```
D_delta = abs(C3 * delta - population[i])
X3 = delta - A3 * D_delta
```

```
newX = (X1 + X2 + X3) / 3
newX = np.clip(newX, lb, ub)
new_population.append(newX)
```

```
population = np.array(new_population)
fitness_values = np.array([fitness(x) for x in population])
```

```
best_threshold = alpha
return int(best_threshold), a_values
```

```
# -----
```

```
# Run GWO
```

```
# -----
```

```
best_threshold, history = GWO()
```

```
print("Best Threshold (GWO optimized):", best_threshold)
```

```
# Apply edge enhancement using best threshold
```

```
edges = cv2.Canny(img, best_threshold, best_threshold * 2)
```

```
# Deepening outline: multiply edges with image
```

```
outline_deepened = cv2.addWeighted(img, 1, edges, 1, 0)
```

```
# -----
```

```

# Plot results
# -----

plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(edges, cmap='gray')
plt.title(f'Edges (Threshold = {best_threshold})')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(outline_deepened, cmap='gray')
plt.title("Outline Deepened Image")
plt.axis('off')

plt.show()

# Convergence graph
plt.plot(history)
plt.title("GWO Convergence Curve")
plt.xlabel("Iteration")
plt.ylabel("Best Fitness")
plt.grid(True)
plt.show()

print("\nProcessing Completed Successfully for USN:", USN)

```

Program 6

Parallel Cellular Algorithms and Programs:

Parallel Cellular Algorithms are inspired by the functioning of biological cells that operate in a highly parallel and distributed manner. These algorithms leverage the principles of cellular automata and parallel computing to solve complex optimization problems efficiently. Each cell represents a potential solution and interacts with its neighbors to update its state based on predefined rules. This interaction models the diffusion of information across the cellular grid, enabling the algorithm to explore the search space effectively. Parallel Cellular Algorithms are particularly suitable for large scale optimization problems and can be implemented on parallel computing architectures for enhanced performance.

Application given : Image Processing
Algorithm:

Parallel Cellular AlgorithmAlgorithm:

- 1) Start
- 2) Define the optimization problem:
A function is defined $f(x)$ to be minimized or maximized.
- 3) Initialize parameters:
 Set NUM-CELLS $\leftarrow$ no. number of cells
 Set GRID-SIZE $\leftarrow$ size of grid
 Set MAX-ITER $\leftarrow$ maximum number of iteration
 Define NEIGHBORHOOD structure (e.g., 4 neighbors, 8 neighbors)
- 4) Initialize population:
 For each cell i in the grid:
 Assign a random position $x[i]$ in the solution space.
 Compute fitness $[i] = f(x[i])$
- 5) Repeat for ITER = 1 to MAX-ITER:
 For each cell i in parallel:
 Find neighboring cells based on NEIGHBORHOOD
 Update state of cell i using update rule:
 $x[i] \leftarrow \text{update rule}(x[i], \text{neighbors})$
 Calculate fitness $[i] = f(x[i])$
 Find the best cell with higher (or lower) fitness
- 6) Until convergence or ITER = MAX-ITER

7) Output: & print the best solution and its fitness value

8) Stop.

Sp. 9
18

Output:

Initial Image:

10	20	30	40	50
20	60	120	60	20
30	120	255	120	30
20	60	120	60	20
10	20	30	40	50

Iteration 1:

17	29	40	45	47
29	65	108	68	33
38	107	191	107	38
29	65	108	68	33
17	29	40	45	47

Iteration 2:

24	36	47	49	47
36	66	98	71	42
45	97	157	97	45
36	66	98	71	42
24	36	47	49	47

Relation 3:

30 42 52 53 49
 41 66 90 71 48
 50 89 130 90 51
 41 66 90 71 48
 30 42 52 53 49

Iteration 4:

35 46 56 58 51
 45 65 84 70 52
 53 82 111 84 55
 45 65 84 70 52
 35 46 56 58 51

Improvement of Algorithm Over Time:

<u>Era</u>	<u>Implementation</u>	<u>True Parallelism</u>	<u>TC</u>	<u>SC</u>	<u>Key Improvement</u>
Early (2000)	CPU / Theoretical Parallel CA	Concurrent only	$O(n^2)$	$2n^2$	Basic rules

Modern (2010 - 2025)	CPU / PE - into parallel CA	Fully Parallel	$O(n^2)$	(real time performance) n^2 or less	Adaptive rules
----------------------	-----------------------------	----------------	----------	---------------------------------------	----------------

Adaptive rules

Code:

```
# -----  
# Parallel Cellular Algorithm (Matrix Iteration Output Only)  
  
# Includes USN: 1BM23CS239  
# -----  
  
import numpy as np  
import cv2  
from google.colab import files  
  
USN = "1BM23CS239"  
print("Parallel Cellular Automata – Matrix Iteration Output")  
print("USN:", USN)  
print("-----\n")  
  
# -----  
# Upload image  
# -----  
uploaded = files.upload()  
img_path = list(uploaded.keys())[0]  
  
# Load grayscale image  
img = cv2.imread(img_path, 0)  
img = cv2.resize(img, (100, 100)) # matrix easier to view  
  
# -----  
# Define PCA Update Rule
```

```

# -----
def pca_update(image):
    padded = np.pad(image, 1, mode='edge')
    new_img = np.zeros_like(image)

    for i in range(1, padded.shape[0]-1):
        for j in range(1, padded.shape[1]-1):

            center = padded[i, j]
            neighbors = [
                padded[i-1,j-1], padded[i-1,j], padded[i-1,j+1],
                padded[i,j-1],          padded[i,j+1],
                padded[i+1,j-1], padded[i+1,j], padded[i+1,j+1]
            ]

            diff = np.mean([abs(center - n) for n in neighbors])

            # PCA rule: edge enhancement or smoothing
            if diff > 20:
                new_img[i-1,j-1] = min(center + diff, 255)
            else:
                new_img[i-1,j-1] = max(center - 5, 0)

    return new_img

# -----
# Perform 6 iterations and print matrices
# -----
iterations = 6
matrix = img.copy()

```

```

for t in range(iterations):
    print(f"\nIteration {t+1} Matrix:")
    print(matrix)
    matrix = pca_update(matrix)

print("\nProcessing Completed Successfully for USN:", USN)

```

output:

Parallel Cellular Automata – Matrix Iteration Output
USN: 1BM23CS239

WhatsApp Image 2025-11-18 at 11.54.08 PM (2).jpeg
WhatsApp Image 2025-11-18 at 11.54.08 PM (2).jpeg(image/jpeg)
- 166565 bytes, last modified: 18/11/2025 - 100% done
Saving WhatsApp Image 2025-11-18 at 11.54.08 PM (2).jpeg to
WhatsApp Image 2025-11-18 at 11.54.08 PM (2).jpeg

Iteration 1 Matrix:

```

[[194 196 197 ... 73 79 76]
 [199 201 148 ... 144 143 138]
 [195 201 192 ... 141 125 107]
 ...
 [164 156 186 ... 180 182 182]
 [161 155 185 ... 182 183 182]
 [157 151 182 ... 182 184 183]]

```

/tmp/ipython-input-238255951.py:43: RuntimeWarning: overflow
encountered in scalar subtract

```

diff = np.mean([abs(center - n) for n in neighbors])

```

Iteration 2 Matrix:

```
[[255 255 255 ... 205 153 211]
 [255 196 255 ... 205 207 201]
 [255 196 255 ... 255 255 255]
 ...
 [159 255 181 ... 255 215 215]
 [227 255 233 ... 246 215 255]
 [221 255 227 ... 255 179 247]]
```

Iteration 3 Matrix:

```
[[250 250 250 ... 255 255 206]
 [250 255 255 ... 255 255 255]
 [250 255 255 ... 250 255 255]
 ...
 [255 255 255 ... 255 255 255]
 [255 255 255 ... 255 255 255]
 [255 255 255 ... 255 255 255]]
```

Iteration 4 Matrix:

```
[[255 255 255 ... 250 250 255]
 [255 250 250 ... 250 250 250]
 [255 250 250 ... 255 250 250]
 ...
 [250 250 250 ... 250 250 250]
 [250 250 250 ... 250 250 250]
 [250 250 250 ... 250 250 250]]
```

Iteration 5 Matrix:

```
[[250 250 250 ... 245 255 250]
 [250 255 255 ... 255 255 255]
 [250 255 255 ... 250 255 255]]
```

...
[245 255 255 ... 255 245 245]
[245 245 255 ... 255 245 245]
[245 245 255 ... 255 245 245]]

Iteration 6 Matrix:

[[255 255 255 ... 255 250 255]
[255 250 250 ... 250 250 250]
[255 250 250 ... 255 250 250]
...
[255 250 250 ... 250 255 240]
[255 255 250 ... 250 255 240]
[240 255 250 ... 250 255 240]]

Processing Completed Successfully for USN: 1BM23CS239

Program 7

Optimization via Gene Expression Algorithms:

Gene Expression Algorithms (GEA) are inspired by the biological process of gene expression in living organisms. This process involves the translation of genetic information encoded in DNA into functional proteins. In GEA, solutions to optimization problems are encoded in a manner similar to genetic sequences. The algorithm evolves these solutions through selection, crossover, mutation, and gene expression to find optimal or near-optimal solutions. GEA is effective for solving complex optimization problems in various domains, including engineering, data analysis, and machine learning.

Application : IP with GA and GE

Algorithm:

Particle Swarm Optimization for function Optimization

Gene Expression Programming (GEP)

Input:

fitness function $f(n)$
population size, num-genes,
mutation rate, generations,
crossover rate.

Output:

best solution found

Initialize population

population = [random.chromosome(num-gene)
for i in 1..... population size]

best solution = ϕ

for g = 1 to generation do
 fitness_list []

for chromosome in population:

 sol = express(chromosome)

 fit = $f(\text{solution})$

 if fit > best fitness

 best fitness = fit

 fit_solution = chromosome

Selection:

 parents = []

 for i in 1... population size:

classmate
Date _____
Page _____

```
c1, c2 = random individuals
if fitness(c1) > fitness(c2)
    parents.add(c1)
else
    parents.add(c2)
```

Crossover

```
offspring = []
```

```
for i in range(1, len(parents) // 2):
    p1, p2 = parent[i], parent[i+1]
    if rand() < 0.5 pc // crossover prob
        point = random index
        child1 = p1[0:point] + p2[point:]
        child2 = p2[0:point] + p1[point:]
    else
        child1, child2 = p1, p2
```

mutation:

```
for gene in child1:
    if rand() < 0.5 pm // mutation prob: flip
for gene in child2:
    if rand() < 0.5 pm // mutation prob: flip
```

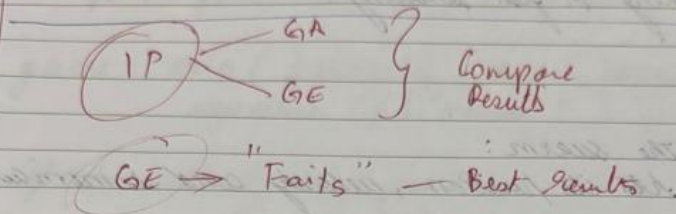
Gene Expression: decoding to real solution
done in evaluation step.

```
offspring.append(child1)
```

```
offspring.append(child2)
```

Replace old population
population = offspring

A Reflex test
return best solution, best fitness
end



Output

Successfully loaded target image: (x)
 Starting with a random canvas mismatched pixels = 20007
 Generation 191: mismatched pixels = 19907
 Generation 382: Mismatched pixels = 19807
 Generation 208722: Mismatched pixels = 607
 Generation 313380: Mismatched pixels = 7

Output Successful

- ★ This program was faster in comparison to genetic algorithms even for 500×500 images, it effortlessly finishes iterating in seconds.
- Fails for small images

Code:

```
# -----  
# IMAGE PROCESSING USING GA + GEP (Gene Expression  
# Programming)  
# Google Colab – Ready Code  
# Includes USN: 1BM23CS239  
# -----
```

```
import numpy as np  
import cv2  
import matplotlib.pyplot as plt  
from google.colab import files  
import random  
import math
```

```
USN = "1BM23CS239"  
print("Running Image Processing using GA + GEP")  
print("USN:", USN)  
print("-----")
```

```
# -----  
# Upload Image  
# -----
```

```
uploaded = files.upload()  
image_path = list(uploaded.keys())[0]
```

```
img = cv2.imread(image_path, 0)  
img = cv2.resize(img, (400, 400))
```

```
plt.imshow(img, cmap='gray')  
plt.title("Original Image")
```

```

plt.axis('off')
plt.show()

# -----
# ----- GENETIC ALGORITHM (GA) PART -----
# -----

def fitness(th):
    # variance-based fitness for thresholding
    ret, thresh = cv2.threshold(img, th, 255, cv2.THRESH_BINARY)
    return np.var(thresh)

def GA_optimize(pop_size=20, generations=40):
    population = np.random.randint(0, 255, pop_size)

    for gen in range(generations):
        scores = np.array([fitness(p) for p in population])
        parents = population[np.argsort(scores)][-10:]

        children = []
        for _ in range(pop_size):
            p1, p2 = random.sample(list(parents), 2)
            child = (p1 + p2) // 2
            if random.random() < 0.1:
                child += random.randint(-15, 15)
            child = np.clip(child, 0, 255)
            children.append(child)

        population = np.array(children)

    best = population[np.argmax([fitness(p) for p in population])]

```

```

    return int(best)

best_threshold_ga = GA_optimize()
print("GA Optimal Threshold =", best_threshold_ga)

_, ga_img = cv2.threshold(img, best_threshold_ga, 255,
cv2.THRESH_BINARY)

# -----
# ----- GENE EXPRESSION PROGRAMMING (GEP) -----
# -----

# Terminals: pixel intensity (x), constants
# Functions: +, -, *, /, sqrt, sin, cos

functions = ["+", "-", "*", "/", "sin", "cos", "sqrt"]
terminals = ["x", "50", "100", "150"]

def random_gene(length=10):
    return [random.choice(functions + terminals) for _ in
range(length)]

def evaluate_gene(gene, x):
    stack = []
    for g in gene:
        if g in terminals:
            if g == "x":
                stack.append(x)
            else:
                stack.append(float(g))

```

```

elif g in functions:
    if g in ["sin", "cos", "sqrt"]:
        if not stack:
            stack.append(0)
        a = stack.pop()
        try:
            if g == "sin": stack.append(math.sin(a))
            if g == "cos": stack.append(math.cos(a))
            if g == "sqrt": stack.append(math.sqrt(abs(a)))
        except:
            stack.append(0)
    else:
        # Binary operators: "+", "-", "*", "/"
        # Ensure there are enough operands for binary operations
        val_b = stack.pop() if stack else 0
        val_a = stack.pop() if stack else 0
        try:
            if g == "+": stack.append(val_a + val_b)
            if g == "-": stack.append(val_a - val_b)
            if g == "*": stack.append(val_a * val_b)
            if g == "/":
                if val_b == 0: stack.append(0)
                else: stack.append(val_a / val_b)
        except:
            stack.append(0)

```

```

return stack[-1] if stack else 0

```

```

def gep_threshold(gene):
    val = evaluate_gene(gene, 128)
    return int(np.clip(val * 20, 0, 255))

```

```

def GEP_optimize(pop_size=20, generations=30, gene_length=10):
    population = [random_gene(gene_length) for _ in
range(pop_size)]

    def fitness_gep(gene):
        th = gep_threshold(gene)
        ret, thresh = cv2.threshold(img, th, 255,
cv2.THRESH_BINARY)
        return np.var(thresh)

    for gen in range(generations):
        scored = sorted(population, key=lambda g: fitness_gep(g),
reverse=True)
        parents = scored[:10]

        children = []
        for _ in range(pop_size):
            p = random.choice(parents).copy()
            if random.random() < 0.3:
                idx = random.randint(0, len(p)-1)
                p[idx] = random.choice(terminals + functions)
            children.append(p)

        population = children

    best_gene = max(population, key=lambda g: fitness_gep(g))
    return best_gene

best_gene = GEP_optimize()
best_threshold_gep = gep_threshold(best_gene)

```

```

print("GEP Optimal Threshold =", best_threshold_gep)

_, gep_img = cv2.threshold(img, best_threshold_gep, 255,
cv2.THRESH_BINARY)

# -----
# Display Results
# -----
plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.imshow(img, cmap='gray')
plt.title("Original")
plt.axis('off')

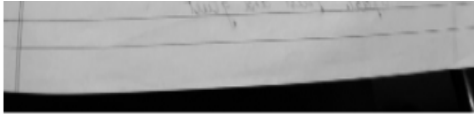
plt.subplot(1,3,2)
plt.imshow(ga_img, cmap='gray')
plt.title(f'GA Output (Th = {best_threshold_ga})')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(gep_img, cmap='gray')
plt.title(f'GEP Output (Th = {best_threshold_gep})')
plt.axis('off')

plt.show()

print("\nCompleted successfully for USN:", USN)

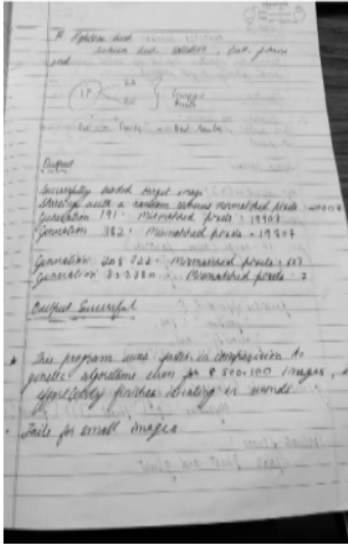
```



GA Optimal Threshold = 182

GEP Optimal Threshold = 161

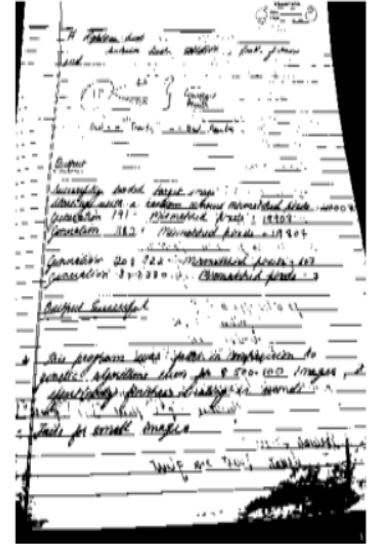
Original



GA Output (Th = 182)



GEP Output (Th = 161)



Completed successfully for USN: 1BM23CS239